Algorithmic Efficiency & Recursion Toolkit (AERT)

Course: Data Structures (ETCCDS202)

Unit: Foundations & Algorithmic Analysis

Student Name: Jagrit Jaggi

Roll No: 2501730191

Course: B-tech CSE AI/Ml

Section: A

1. Abstract Data Type (ADT) and Stack

An Abstract Data Type (ADT) is a logical description of a data structure that defines what operations can be performed, without specifying how they are implemented internally.

For example, a Stack ADT defines operations such as:

- push(x) → insert element at the top
- pop() → remove the top element
- peek() → view the top element
- is_empty() → check whether stack is empty
- size() → return number of elements

The stack follows the LIFO principle (Last In First Out). The element that is inserted last is removed first.

Q) Why Stack Operations Are O(1)?

Most stack operations such as push and pop operate only on the top element of the stack. Since the operation does not depend on the number of elements in the stack, the time required remains constant.

Therefore:

Time Complexity = O(1)

In this toolkit, the Stack ADT is used to store the Tower of Hanoi moves, demonstrating how stacks can track operations during recursion.

2. Recursive Factorial

The factorial of a number n is defined as:

$n! = n \times (n - 1)!$

with the base case:

$0! = 1$

The recursive algorithm repeatedly multiplies the number by the factorial of the previous number until it reaches the base case.

Example

$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

Time Complexity

The function calls itself n times, therefore:

Time Complexity:
O(n)

Space Complexity

Recursion uses the call stack, which stores each function call until the base case is reached.

Space Complexity:
O(n)

Best, Average, and Worst Cases

Since the same number of recursive calls are made for a given n:

Best Case = O(n)
Average Case = O(n)
Worst Case = O(n)

3. Fibonacci Sequence

The Fibonacci sequence is defined as:

$F(n) = F(n - 1) + F(n - 2)$

with base cases:

$F(0) = 0$
$F(1) = 1$

Two recursive versions were implemented in the toolkit.

3.1 Naive Recursive Fibonacci

The naive method directly follows the mathematical definition and recursively calculates previous values.

Example

$F(5) = F(4) + F(3)$

Each call further expands into more recursive calls.

Time Complexity

The algorithm repeatedly recalculates the same values many times.

Time Complexity:
$O(2^n)$

Space Complexity

The recursion depth is at most n. O(n)

Why Naive Fibonacci Is Slow

The naive version recomputes the same Fibonacci values multiple times.

For example:

F(5) requires F(4) and F(3)
F(4) again requires F(3) and F(2)

This repeated work causes the number of recursive calls to grow exponentially, making the algorithm very slow for larger values of n.

3.2 Memorized Fibonacci (Dynamic Programming)

Memorization stores previously computed results so they do not need to be recalculated.

A dictionary (memo) is used to store values that have already been computed.

When the function needs a value that already exists in the memo, it directly returns it instead of recomputing.

Time Complexity

Each Fibonacci number is calculated only once.

Time Complexity:
O(n)

Space Complexity

The memo dictionary stores n values. O(n)

Memorization greatly improves efficiency compared to the naive recursive approach.

<u>4. Tower of Hanoi</u>

The Tower of Hanoi is a classic recursive problem involving three rods and n disks.

The goal is to move all disks from the source rod to the destination rod while following these rules:

1. Only one disk can be moved at a time.
2. A larger disk cannot be placed on a smaller disk.

Recursive Idea

To move n disks:

1. Move n − 1 disks from source to auxiliary.
2. Move the largest disk to destination.
3. Move n − 1 disks from auxiliary to destination.

Number of Moves

The total number of moves required follows the formula:

Moves = $2^n - 1$

Example:

n = 3
Moves = $2^3 - 1 = 7$ moves

Time Complexity

Each recursive step splits into two smaller problems.

Time Complexity:
$O(2^n)$

Space Complexity

The recursion depth is equal to the number of disks.

Space Complexity:
O(n)

<u>5. Recursive Binary Search</u>

Binary search is an efficient searching algorithm that works on sorted arrays.

It repeatedly divides the array into halves to locate the target element.

Steps

1. Find the middle element.

2. If the key equals the middle element, return the index.
3. If the key is smaller, search the left half.
4. If the key is larger, search the right half.

Recurrence Relation

The recursive relation for binary search is:

$T(n) = T(n/2) + O(1)$

This means the problem size is reduced by half at each step.

Solving this recurrence gives:

Time Complexity = $O(\log n)$

Best, Average, and Worst Cases

Best Case:
Element found at middle immediately.
$O(1)$

Average Case:
Element found after several divisions.
$O(\log n)$

Worst Case:
Element not present or found after maximum divisions.
$O(\log n)$

Binary search is much faster than linear search for large sorted datasets.

6. Algorithm Paradigms Reflection

Different algorithms use different problem-solving strategies called algorithm paradigms.

7. Short Paradigm Reflection

Divide and Conquer

Divide and Conquer works by breaking a problem into smaller subproblems, solving them independently, and combining the results.

Binary Search follows this paradigm because the array is divided into two halves at each step.

Real Life Example

Searching for a word in a dictionary by opening the middle page and narrowing the search.

Dynamic Programming

Dynamic Programming solves problems by storing results of previously solved subproblems.

The memorized Fibonacci algorithm uses dynamic programming because it saves already computed Fibonacci values and reuses them.

Real Life Example

Remembering solutions to previously solved math problems so they do not need to be solved again.

Greedy Algorithm

Greedy algorithms choose the locally optimal solution at each step with the hope of reaching the global optimum.

Real Life Example

Making change using the largest denomination coins first.

For example, to give ₹87 using the largest currency notes available.

7. Conclusion

In this assignment, a small Python toolkit was developed to demonstrate important concepts in data structures and algorithm analysis.

The project implemented:

- Stack ADT operations
- Recursive factorial
- Naive and memorized Fibonacci
- Tower of Hanoi recursion
- Recursive binary search

The assignment also analysed the time and space complexity of these algorithms using Big-O notation and explored different algorithm paradigms such as Divide and Conquer, Dynamic Programming, and Greedy methods.

This toolkit helps demonstrate how recursion works, how algorithm efficiency can vary greatly depending on implementation, and why algorithm analysis is important in computer science.